GCPS – ATIVIDADE I - N2

KELWIN GAMEIRO DOS SANTOS E JOÃO FILIPI TISSI

Domínio 1: Sistema Crítico com Certificação de Segurança

1. Escolha dos modelos por parte do sistema e justificativa

Optamos por dividir o sistema em duas partes, para melhor seguir as premissas apresentadas por cada parte identificada. No modelo que escolhemos, o núcleo certificável (sinalização, intertravamento e frenagem) segue o Modelo V. Os componentes periféricos (painéis e aplicativo) seguem Scrum combinado com XP.

No núcleo, a escolha do Modelo V se justifica pelos critérios a seguir:

- **Estabilidade dos requisitos**: Os requisitos de segurança já estão bem estabelecidos e seguem normas legais já consolidadas, precisando ser validados por um organismo independente e seguindo uma certificação legal. Isso caracteriza a especificação fechada sobre a qual o Modelo V trabalha. Para a Criticidade e regulamentação, o Modelo V oferece exatamente a rastreabilidade entre requisito e teste e as evidências para certificação.
- **Natureza do risco principal**: o risco é de segurança, já que uma falha nos cruzamentos ou na frenagem em produção é inaceitável. Esse é o risco que o Modelo V foi feito para controlar.
- **Tolerância a retrabalho tardio**: Como a tolerância é baixa, o Modelo V lida com isso estabilizando cada fase antes de avançar.
- **Frequência de entrega**: o núcleo pode ser entregue ao final do ciclo, após os testes de aceitação, o que é compatível com o modelo.

Já na periferia, optamos por uma mescla de Scrum + XP. Os critérios para a escolha são apresentados a seguir:

- **Requisitos instáveis**: os painéis e o aplicativo recebem pedidos frequentes de mudança da prefeitura e dos usuários. Scrum e XP foram feitos para requisitos para serem flexíveis, e se adaptarem à esse cenário. Esses componentes não passam pela certificação. A qualidade pode vir dos testes automatizados do XP e da inspeção pela Definition of Done do Scrum.
- **Alta frequência de entrega**: Como a prefeitura quer ver resultados com frequência, o Scrum e XP oferecem incrementos curtos e regulares, cumprindo a demanda por resultados visíveis e constantes a curto prazo.
- **Natureza do risco principal**: a periferia combina dois riscos. O risco de negócio, de construir o que a prefeitura não quer, é controlado pelo Scrum. O risco técnico, de código instável e integração difícil num código que mudará muito, é controlado pelo XP.

- **Tolerância a retrabalho tardio**: a periferia precisa absorver mudanças constantes. O Scrum absorve retrabalho no backlog, e a refatoração contínua do XP torna a mudança barata.

2. Elementos do Scrum e do XP na periferia

A escolha pela combinação dos dois modelos foi erguida critério a critério, compreendendo necessidades particulares e adotando de cada modelo o elemento que atende ao que é exigido.

Do Scrum: Sprints, Product Owner, Product Backlog e Sprint Review.

- **Disponibilidade do cliente**: a prefeitura atua como Product Owner, disponível e com autoridade sobre o backlog. Esse é o modo de participação que o Scrum estrutura.
- **Estabilidade dos requisitos**: o Scrum permite que as prioridades mudem entre Sprints enquanto protege o objetivo de cada Sprint. Isso dá à periferia flexibilidade e, ao mesmo tempo, previsibilidade para o trabalho que depende da interface com o núcleo.
- **Tamanho da equipe**: os 5 desenvolvedores web/mobile formam uma equipe pequena, adequada ao Scrum.
- **Natureza do risco**: a Sprint Review mostra o incremento à prefeitura a cada Sprint. Isso permite corrigir a direção cedo e controlar o risco de negócio.

Do XP: TDD, testes de aceitação automatizados, integração contínua, refatoração contínua e programação em pares.

- **Natureza do risco:** TDD e integração contínua atacam o risco técnico de código instável, débito técnico e integração difícil, inclusive com a API do núcleo.
- **Tolerância a retrabalho tardio**: refatoração contínua e testes de regressão tornam a mudança barata, o que é necessário num código que mudará muito.
- **Necessidade de verificação e validação**: o XP oferece verificação alta e automatizada, adequada a componentes que exigem qualidade mas não exigem evidência formal.
- **Tamanho/maturidade da equipe**: o XP pede uma equipe pequena e com disciplina técnica. Como metade dos desenvolvedores é recém-contratada, a programação em pares é a forma de construir essa disciplina.

Para frequência de entrega, adotamos Sprints de 2 semanas, por entender que esse intervalo seria capaz de saciar ambos os modelos.

3. Manutenção da matriz de rastreabilidade entre requisito e teste

A matriz atende ao critério de criticidade/regulação: ela é a evidência que o auditor exige.

- **Conteúdo**: cada requisito de segurança recebe um identificador, por exemplo RS-012.
- **Ligações**: esse identificador é ligado ao elemento de projeto, ao código e ao caso de teste que o verifica, com o resultado registrado. A ligação vale nos dois sentidos.

O critério de verificação e validação formal define quando a matriz é preenchida. Os casos de teste entram junto com a fase que verificam, antes da codificação:

No Modelo V, cada fase de projeto tem um nível de teste correspondente. Os requisitos são verificados pelo teste de aceitação/validação, a arquitetura pelo teste de integração e o projeto detalhado pelo teste de unidade.

As práticas de testes automatizados e integração contínua usadas no núcleo geram resultados que são anexados à matriz como evidência. Elas complementam as evidências formais de cada fase.

A responsabilidade segue o critério de natureza do risco principal. Como o risco é de segurança, o engenheiro de segurança é o dono da matriz.

O engenheiro de segurança aprova o conteúdo da matriz e garante que todo requisito tenha um teste correspondente, enquanto os analistas de testes mantêm os casos de teste e as evidências. Por fim, os engenheiros embarcados registram as ligações com projeto e código.

No núcleo:

Os engenheiros embarcados cuidam do projeto e da codificação. Os Analistas de testes planejam e executam cada nível de teste, atendendo à necessidade de verificação formal.

O engenheiro de segurança responde pela matriz e tem poder de veto sobre mudanças que afetem a segurança, porque esse é o risco principal do núcleo, enquanto o CCB do Núcleo é formado pelo engenheiro de segurança, pelo líder técnico do núcleo, por um representante dos testes, pelo gerente de configuração e por um representante do consórcio.

Na periferia:

Product Owner é um representante da prefeitura, com autoridade sobre o backlog.

O Scrum Master é um dos desenvolvedores mais experientes ou o gerente de projeto.

O Time de desenvolvimento são os 5 desenvolvedores web/mobile, trabalhando em pares para desenvolver a disciplina técnica que o XP pressupõe.

Enquanto na interface, o engenheiro de segurança também revisa a interface entre núcleo e periferia, porque ela recebe dados de uma parte de criticidade alta.

4. Como e quando ocorrem as baselines

No núcleo, a baseline é formal e estabelecida ao final de cada fase do V: requisitos, arquitetura, projeto detalhado, código e cada nível de teste.

- **Por que por fase**: a tolerância a retrabalho tardio é baixa, então cada fase é consolidada antes de a seguinte começar.
- **Como mudam:** alterações só ocorrem via CCB, com análise de impacto.
- **Por que funciona:** a alta estabilidade dos requisitos torna esse controle eficiente, porque mudanças são raras.

Na periferia, a baseline é estabelecida a cada Sprint.

- A cada Sprint o incremento é aprovado na review recebe uma versão marcada. Isso acompanha a entrega de um incremento utilizável por sprint.
- Entre as Sprints: a integração contínua do XP gera versões continuamente, acompanhando a frequência de entrega alta e a necessidade de mudanças baratas.
- Nível de formalidade: essas versões ficam registradas no repositório. A formalidade é proporcional à criticidade dos componentes.

5. Tratamento da interface entre núcleo e periferia

A interface, por exemplo a API de posição dos veículos, segue o rigor do Modelo V. Três critérios justificam isso:

- **Tolerância a retrabalho tardio**: uma alteração na interface é uma alteração no núcleo e afeta suas baselines de projeto e teste. Por isso ela precisa ser controlada.
- **Criticidade/regulação**: os dados têm origem no núcleo certificável. Por isso o contrato faz parte da rastreabilidade e das evidências.
- **Estabilidade dos requisitos**: o contrato é tratado como especificação fechada, mantido em baseline e alterado apenas via CCB.

A periferia mantém seu ritmo de frequência de entrega. A cada Sprint, ela desenvolve sobre a versão vigente do contrato, e os testes automatizados do XP verificam a compatibilidade a cada build.

A integração com novas versões do núcleo acontece em pontos de sincronização definidos. Esses pontos são as entregas do núcleo ao final de suas fases, alinhadas a Sprint Reviews específicas.

6. Riscos de misturar modelos e mitigação

Os riscos aparecem nos critérios em que núcleo e periferia têm necessidades diferentes.

1. Estabilidade dos requisitos.

- **Risco**: a periferia pedir mudanças na interface no meio de uma Sprint.
- **Mitigação**: a periferia protege o objetivo da Sprint, como prevê o Scrum. Pedidos de mudança na interface vão para o backlog e depois para o CCB do núcleo, onde são incluídos numa entrega planejada.

2. Frequência de entrega.

- **Risco**: a periferia entrega a cada 2 semanas e o núcleo ao final das fases. Isso poderia bloquear a periferia.
- **Mitigação**: a periferia trabalha sempre sobre a versão do contrato em baseline, e a integração com novas versões ocorre nos pontos de sincronização planejados.

3. Criticidade e verificação formal.

- **Risco**: as práticas do XP aplicadas no núcleo enfraquecerem a produção de evidências.

- **Mitigação**: fica estabelecido que testes automatizados e integração contínua alimentam a matriz de rastreabilidade, somando-se às evidências formais de cada fase do V.

4. Tolerância a retrabalho tardio.

- **Risco**: a equipe periférica, acostumada a mudanças baratas, tratar o núcleo da mesma forma.
- **Mitigação**: o código e o contrato do núcleo ficam sob controle do CCB, e alterações nessa parte só são feitas com sua aprovação.

5. Disponibilidade do cliente.

- **Risco**: a prefeitura, ativa como PO na periferia, levar demandas diretamente ao núcleo.
- **Mitigação**: o poder de priorização do PO se aplica ao backlog da periferia. Demandas que envolvem o núcleo seguem o fluxo formal de mudança via CCB, com participação do consórcio.

6. Maturidade da equipe.

- **Risco**: a disciplina técnica que o XP pressupõe ainda estar em formação, já que metade da equipe periférica é recém-contratada.
- **Mitigação**: programação em pares e TDD são obrigatórios, e a integração contínua só incorpora código acompanhado de testes.

Domínio 2: Produto Digital em Evolução com Componente Regulado

1. Escolha dos modelos

Para este projeto, daria pra escolher Scrum, XP e Modelo V, pois cada modelo atende a uma necessidade diferente.

Scrum: Organizar as tarefas, escolher as funções mais importantes e planejar as entregas.

XP: Melhorar o código, reduzir erros e ajudar os programadores iniciantes a aprender.

Modelo V: Somente no módulo de emissão de NFC-e, pois ele precisa seguir regras fiscais e passar por testes e homologação antes de ser liberado.

O Scrum organiza o trabalho, o XP ajuda a equipe a programar com mais qualidade e o Modelo V aumenta o controle sobre a parte fiscal.

2. Justificativa pelos critérios

2.1 Estabilidade dos requisitos

Os clientes pedem novas funções toda semana. O Scrum ajuda a organizar os pedidos, e o XP permite melhorar o sistema aos poucos. Já o módulo de NFC-e usará o Modelo V, com testes planejados desde o início.

2.2 Regulamentação

Erros na plataforma podem prejudicar as vendas e o estoque. O XP ajuda a encontrar esses erros com testes frequentes. No módulo de NFC-e, o Modelo V ajuda a verificar as regras fiscais e evitar rejeições e multas.

2.3 Necessidade de verificação e validação formal

O XP permite testar o código durante o desenvolvimento, enquanto o Scrum ajuda a avaliar as entregas a cada Sprint. No módulo de NFC-e, serão feitos testes e uma aprovação no ambiente de testes da Secretaria da Fazenda.

2.4 Frequência de entrega

Como os clientes pedem mudanças frequentes, o Scrum organiza as entregas e o XP ajuda a manter o sistema funcionando. O módulo fiscal terá testes e aprovação próprios antes de ser liberado.

2.6 Tamanho e maturidade da equipe

A equipe tem seis pessoas: dois programadores experientes e quatro iniciantes. O Scrum organiza as tarefas, e o XP permite que os mais experientes ajudem os iniciantes. Assim, a equipe aprende e reduz erros.

3. Práticas do XP que serão adotadas primeiro

Como quatro dos seis programadores são iniciantes e o sistema possui problemas antigos, as práticas do XP serão adotadas aos poucos, nesta ordem:

1. **Testes automatizados**: criar testes para as funções mais importantes, principalmente aquelas que podem apresentar erros após uma atualização.
2. **Integração contínua**: juntar as alterações ao código principal com frequência e executar os testes para encontrar problemas rapidamente.
3. **Programação em duplas**: colocar um programador experiente com um iniciante em tarefas importantes, ajudando na aprendizagem e na redução de erros.
4. **Refatoração contínua**: melhorar a organização do código antigo sem alterar o resultado esperado das funções.
5. **Pequenas alterações**: evitar mudanças muito grandes de uma só vez, facilitando os testes e a correção de problemas.

Essas práticas serão iniciadas pelas partes mais importantes do sistema e ampliadas conforme a equipe ganhar experiência.

4. Como o módulo de NFC-e funcionará dentro das Sprints

O módulo de NFC-e continuará fazendo parte da plataforma, mas terá um planejamento próprio de testes e aprovação.

O Scrum organizará as tarefas em Sprints, enquanto o Modelo V será responsável por orientar os testes específicos do módulo fiscal.

As etapas serão:

1. Registrar os requisitos fiscais.
2. Planejar os testes para cada requisito.
3. Desenvolver o código e realizar os testes de cada parte.
4. Testar a comunicação entre as partes do sistema.
5. Realizar os testes de homologação no ambiente da Secretaria da Fazenda.
6. Corrigir os erros encontrados e repetir os testes necessários.
7. Liberar o módulo somente depois de cumprir as exigências de homologação.

Uma tarefa pode ser concluída dentro de uma Sprint sem que o módulo fiscal inteiro esteja pronto para entrar em produção.

Por isso, a definição de pronto será diferente para esse módulo. Além de funcionar e passar nos testes da equipe, ele deverá cumprir os requisitos fiscais e obter a homologação necessária.

5. Como será feita a GCS

A GCS (Gerência de Configuração de Software) será adaptada aos modelos escolhidos.

5.1 Na plataforma geral, com Scrum e XP

- Cada Sprint terá uma versão identificada das funções entregues.
- As alterações serão registradas no repositório de código.
- Os testes serão executados frequentemente.
- As mudanças serão verificadas para evitar que funções antigas parem de funcionar.
- Os registros serão simples, mas suficientes para acompanhar o que foi alterado.
- Nesse caso, a formalidade será média, permitindo controlar as mudanças sem atrasar as entregas.

5.2 No módulo de NFC-e, com Modelo V

- Os requisitos fiscais serão registrados formalmente.
- Cada requisito será relacionado aos testes correspondentes.
- As versões aprovadas serão identificadas e guardadas.
- Os resultados dos testes e da homologação serão registrados.
- As mudanças serão analisadas antes de serem aprovadas.

Nesse módulo, a formalidade será alta, pois é necessário comprovar que as regras fiscais foram atendidas.

6. Problemas que podem surgir ao combinar os modelos

A combinação dos modelos pode gerar dificuldades se cada um usar formas diferentes de controlar as alterações.

Por exemplo, o XP incentiva melhorias frequentes no código, enquanto o Modelo V exige registros e testes planejados. Uma alteração feita sem controle pode afetar uma função fiscal que já havia sido testada.

Para evitar isso, a equipe utilizará o mesmo repositório de código e manterá as versões identificadas. As mudanças comuns seguirão o planejamento das Sprints, enquanto as alterações do módulo fiscal também precisarão passar pelas verificações do Modelo V.

Outro problema pode ser a falta de disponibilidade do dono do produto. Para diminuir esse risco, as prioridades e os critérios para aceitar as funções deverão ser definidos antes das Sprints, sempre que possível.

Por fim, os testes fiscais não poderão ser ignorados para cumprir uma data de entrega. Se houver erros, o módulo só será liberado depois das correções e das verificações necessárias.

7. Conclusão

A combinação de Scrum, XP e Modelo V é a melhor escolha para esse domínio porque cada modelo resolve um problema diferente.

O Scrum organiza as tarefas e as entregas. O XP ajuda a melhorar o código, reduzir erros e preparar os programadores iniciantes. O Modelo V garante um controle maior sobre o módulo de NFC-e, que precisa cumprir regras fiscais e passar por homologação.

Assim, a empresa poderá continuar entregando novas funções aos clientes, melhorar a qualidade do sistema e reduzir os riscos de erros na emissão de notas fiscais.